

I2C Driver Analysis

Introduction

The I2C (Inter-Integrated Circuit) driver is used to provide communication between the STM32F446xx microcontroller and external devices such as sensors, EEPROMs, and LCD modules using only two wires: SDA (Serial Data Line) and SCL (Serial Clock Line). The driver hides the low-level register operations and provides easy-to-use functions for transmitting and receiving data.

Header File (stm32f446xx_i2c_driver.h)

The header file contains all the structures, macros, and function declarations required for the I2C driver.

I2C Configuration Structure

The I2C_Config_t structure stores the configuration parameters of the I2C peripheral.

- I2C_SCLSpeed sets the communication speed.
- I2C_DeviceAddress stores the device address.
- I2C_AckControl enables or disables acknowledgment.

I2C Handle Structure

The I2C_Handle_t structure contains:

- Pointer to the I2C peripheral registers.
- I2C configuration structure.

This makes it possible to manage different I2C peripherals using the same driver functions.

Macros

Some commonly used macros are defined:

- I2C_SCL_SPEED_SM – Standard mode speed (100 kHz)
- I2C_ACK_ENABLE – Enable acknowledgment
- I2C_ACK_DISABLE – Disable acknowledgment
- I2C_ENABLE_SR – Enable repeated start
- I2C_DISABLE_SR – Disable repeated start

These macros improve readability and reduce the use of hard-coded values.

Source File (stm32f446xx_i2c_driver.c)

I2C_PeriClockControl()

This function enables the clock for the I2C peripheral by setting the corresponding bit in the RCC APB1ENR register. The peripheral must receive a clock before it can operate.

I2C_Init()

This function initializes the I2C peripheral.

It performs the following operations:

1. Reads the APB1 clock frequency using `RCC_GetPCLK1Value()`.
2. Calculates the CR2 register value.
3. Calculates the CCR register value for generating the required SCL clock.
4. Calculates and configures the TRISE register.

After these steps, the I2C peripheral is ready for communication.

I2C_PeripheralControl()

This function enables or disables the I2C peripheral by controlling the PE (Peripheral Enable) bit in the CR1 register.

I2C_ManageAcking()

This function enables or disables the ACK bit.

Acknowledgment is important during I2C communication because it tells the sender that the data has been received successfully.

I2C_MasterSendData()

This function sends data from the STM32 (master) to a slave device.

The communication sequence is:

1. Generate START condition.
2. Wait until the START bit is set.
3. Send the slave address with the write bit.
4. Wait for address acknowledgment.
5. Clear the ADDR flag.
6. Send all data bytes one by one.
7. Wait until transmission is complete.
8. Generate STOP condition unless repeated start is selected.

I2C_MasterReceiveData()

This function receives data from a slave device.

The steps followed are:

1. Generate START condition.
2. Send the slave address with the read bit.
3. Wait until the address is acknowledged.
4. Clear the ADDR flag.
5. Read one or multiple bytes from the data register.
6. Disable ACK before receiving the last byte.
7. Generate STOP condition if repeated start is not required.
8. Restore ACK if it was originally enabled.

Static Helper Functions

Several helper functions are declared as static because they are used only inside the driver.

I2C_GenerateStartCondition()

Sets the START bit in CR1 to begin communication.

I2C_GenerateStopCondition()

Sets the STOP bit in CR1 to terminate communication.

I2C_ExecuteAddressPhaseWrite()

Shifts the slave address left by one bit and clears the least significant bit to indicate a write operation before loading it into the data register.

I2C_ExecuteAddressPhaseRead()

Shifts the slave address left by one bit and sets the least significant bit to indicate a read operation before loading it into the data register.

I2C_ClearADDRFlag()

Clears the ADDR flag by reading the SR1 register followed by the SR2 register, as required by the STM32 hardware.

Hardware Abstraction

The driver hides all the low-level register manipulations from the application programmer. Instead of directly accessing CR1, CR2, CCR, TRISE, SR1, SR2, and DR registers, the application only needs to call functions such as:

- I2C_Init()
- I2C_MasterSendData()
- I2C_MasterReceiveData()

This makes the application code simpler, easier to understand, and reusable.

APIs Provided

The driver provides the following APIs:

- I2C_Init() – Initializes the I2C peripheral.
- I2C_PeriClockControl() – Enables or disables the peripheral clock.
- I2C_PeripheralControl() – Enables or disables the I2C peripheral.
- I2C_ManageAcking() – Controls acknowledgment.
- I2C_MasterSendData() – Sends data to a slave device.
- I2C_MasterReceiveData() – Receives data from a slave device.

The I2C driver provides a simple interface for communication between the STM32F446xx microcontroller and external I2C devices. It performs all the required register configurations internally and offers easy-to-use APIs for initialization, data transmission, and data reception. By separating hardware details from application code, the driver improves code readability, portability, and maintainability.